UNIVERSIDAD PRIVADA FRANZ TAMAYO

SEDE EL ALTO

FACULTAD DE INGENIERIA

CARRERA DE INGENIERIA DE SISTEMAS

PROYECTO

"Simulador de Enfrentamiento Directo entre Selecciones de la Copa Mundial de la FIFA 2026 Basado en Red Neuronal"

AUTOR: BRAYAN QUISPE MAMANI

TUTOR: AMERICO ALVAREZ SURCI

EL ALTO - BOLIVIA

2026

ÍNDICE

ÍNDICE 1
CAPÍTULO 1. INTRODUCCIÓN 1
1.1. Planteamiento del problema 1
1.1.1. Identificación de la situación problémica 1
1.1.2. Formulación del problema 3
1.1.3. Problemas específicos 3
1.2. Objetivos 5
1.2.1. Objetivo general 5
1.2.2. Objetivos específicos 5
1.3. Alcances 6
1.4. Límites 6
1.5. Enfoque tecnológico 7
CAPÍTULO 2. MARCO PRÁCTICO 1
2.1. Análisis del dataset 1
2.1.1. Verificación de los archivos CSV del ZIP 3
2.2. Pipeline de implementación propuesto 3
2.3. Limpieza de datos 5
BIBLIOGRAFÍA 8

CAPÍTULO 1. INTRODUCCIÓN

1.1. Planteamiento del problema

Este apartado delimita el problema que el proyecto se propone resolver. El análisis se organiza en tres niveles: primero se describe la situación problémica en el contexto del torneo y de los datos disponibles; luego se formula la pregunta central que orienta el desarrollo; finalmente se desglosan los problemas específicos que el proyecto se propone atacar y que se convierten, uno a uno, en los objetivos del apartado 1.2.

1.1.1. Identificación de la situación problémica

La ampliación del torneo no es un simple ajuste de calendario. Al pasar de 32 a 48 selecciones, el número de emparejamientos posibles pasa de 496 a 2.256, y con él crece el número de preguntas que deben responderse antes de la fase de dieciseisavos de final: ¿con qué probabilidad gana cada cruce? A esa necesidad se suman dos condiciones: las selecciones son muy distintas entre sí y, por lo tanto, los enfrentamientos también lo son.

El Mundial de la FIFA 2026 trae un cambio que no se veía en décadas: por primera vez participan 48 selecciones nacionales. Con tantos equipos en el campo, estimar quién tiene más opciones de quedarse con un partido se vuelve un problema real. Casi siempre se recurre a la opinión de los expertos o a rankings que no terminan de explicar por qué un equipo sería favorito sobre otro.

El dataset "FIFA World Cup 2026 Team Data", publicado en Kaggle, reúne estadísticas acumuladas de esas 48 selecciones: goles, porcentaje de posesión, disparos a puerta, edad promedio del plantel, tarjetas y decenas de métricas ofensivas y defensivas. El dato existe y está ordenado. Lo que falta es una forma sencilla de convertirlo en una respuesta: ¿cuál es la probabilidad de que el equipo A le gane al equipo B?

Hoy esa pregunta se responde de tres maneras, y ninguna resuelve el problema de forma satisfactoria para un entorno académico. La primera es la opinión del especialista: analistas y comentaristas estiman los favoritos a partir de su

experiencia, lo cual aporta contexto pero no es reproducible ni auditable. La segunda son los rankings oficiales, como los que publica la FIFA, que resumen decenas de variables en un solo número, pero ocultan el peso de cada una, se actualizan con poca frecuencia y no responden a un enfrentamiento concreto: dicen qué equipo es mejor en general, no cómo termina el partido entre dos selecciones específicas. La tercera son las cuotas de las casas de apuestas, que sí incorporan información estadística, pero responden también a factores de mercado, a la liquidez de la apuesta y al riesgo que asume el operador, por lo que no pueden usarse como referencia técnica.

El dataset publicado en Kaggle aporta el material cuantitativo para construir una respuesta propia. Sin embargo, su análisis descriptivo revela que la información es agregada por selección y no por partido: cada fila resume la actuación global de un plantel en un torneo, sin fecha, sin rival y sin resultado partido a partido. Es decir, el archivo no contiene confrontaciones directas históricas con un ganador observado, y por tanto tampoco aporta la etiqueta natural que un modelo de clasificación necesita. Además, el número de partidos jugados no es igual para todos los equipos, varía entre 3 y 8, de modo que las métricas acumuladas no son directamente comparables sin una normalización por cantidad de encuentros; y algunas columnas, sobre todo las relativas a la actuación de los porteros, presentan valores vacíos. En consecuencia, los datos disponibles son descriptivos, pero no predictivos por sí solos: falta el procedimiento que los convierta en una estimación de victoria.

Al problema de los datos se suma un problema de método. En el contexto de la materia de Inteligencia Artificial no basta con obtener una respuesta: es necesario mostrar el camino completo que va desde la adquisición del dato hasta la entrega de un modelo evaluado, interpretable y reproducible en un entorno local. Eso implica descomprimir el archivo comprimido, verificar y documentar cada uno de los archivos CSV que contiene, construir el conjunto de pares de enfrentamiento, definir la variable de entrada y la etiqueta, entrenar el modelo y medir su desempeño con métricas explícitas. Ninguna de estas etapas puede omitirse, porque cada una introduce decisiones que se reflejan en el resultado final: un error en la selección de

variables, una etiqueta mal definida o un conjunto de entrenamiento mal construido se propagan hasta la probabilidad que el sistema entrega.

1.1.2. Formulación del problema

De la situación descrita se deriva la pregunta central del proyecto: ¿cómo puede estimarse la probabilidad de victoria de una selección A frente a una selección B, a partir de las estadísticas agregadas disponibles en el dataset de la Copa Mundial de la FIFA 2026, mediante un modelo de aprendizaje automático interpretable que pueda entrenarse y ejecutarse íntegramente en un entorno local reproducible?

La pregunta admite una respuesta acotada y deliberadamente sencilla. En lugar de recurrir a un modelo de alta complejidad, la estimación se plantea sobre una red neuronal de una sola neurona con activación sigmoide, es decir, una regresión logística: la entrada es el vector de diferencias entre las métricas de ambos equipos, la salida es un valor entre 0 y 1 que se interpreta como probabilidad de victoria del equipo A, y el resultado se guarda en formato .keras para ser consultado a través de una API. Esta decisión no es una limitación técnica arbitraria, sino una condición del problema: con una sola neurona, los pesos aprendidos son legibles y se puede explicar cuánto aporta cada métrica a la predicción, algo que un modelo de muchas capas no ofrece.

Para responderla no hace falta un modelo enorme. Una red de una sola neurona con activación sigmoide, es decir una regresión logística, puede bastar para empezar. Se calcula la diferencia vectorial entre las métricas de los dos equipos y la neurona devuelve un valor entre 0 y 1. Ese número se lee como probabilidad de victoria. Sencillo de implementar, rápido de explicar y, sobre todo, transparente: los pesos de la neurona dicen cuánto pesa cada métrica en la decisión.

1.1.3. Problemas específicos

El problema general se descompone en los siguientes problemas específicos, cada uno con su propio entregable y su propia evidencia de avance:

- **Disponibilidad y trazabilidad de los datos.** El dataset se distribuye en un archivo comprimido que debe descomprimirse, inventariarse y documentarse. Es necesario dejar constancia de qué archivos CSV contiene, qué información ofrece cada uno y con qué capturas de pantalla se respalda la revisión, de modo que cualquier lector pueda replicar el análisis.
- **Construcción de la variable de entrada.** Los datos disponibles son valores absolutos por selección, mientras que el enfrentamiento directo exige una medida relativa. Se requiere la definición del vector de diferencias entre dos equipos y su estandarización, para que las cuatro variables utilizadas tengan una escala comparable.
- **Definición de la etiqueta y del conjunto de entrenamiento.** Como no existen confrontaciones históricas etiquetadas en el archivo, las etiquetas deben construirse de forma sintética a partir de los pares de selecciones. Eso exige una regla consistente, una distribución equilibrada de las clases y una separación explícita entre los datos de entrenamiento y los de validación.
- **Entrenamiento y evaluación del modelo.** Se necesita definir la función de pérdida, el optimizador y el número de épocas, y medir el desempeño con métricas explícitas (exactitud, precisión, exhaustividad y matriz de confusión) sobre datos que el modelo no haya visto, para no confundir ajuste con capacidad de generalización.
- **Persistencia y disponibilidad del modelo.** El resultado del entrenamiento debe guardarse en un archivo .keras y quedar disponible para su uso desde una interfaz sencilla, de manera que el pronóstico de un cruce pueda consultarse en segundos sin tener que repetir el entrenamiento.
- **Interpretabilidad del resultado.** La estimación entregada debe poder justificarse: los pesos de la neurona y el vector de diferencias que produce se convierten en la explicación de por qué un equipo es considerado favorito, lo que aporta valor didáctico y evita el uso de una caja negra.
- **Documentación y reproducibilidad del proceso.** El notebook, el entorno de trabajo y las decisiones tomadas deben quedar registrados, junto con las capturas

de la revisión de cada archivo del ZIP, para que el proyecto pueda reproducirse, evaluarse y reutilizarse por un tercero.

Este documento es un perfil de idea de proyecto para la materia de Inteligencia Artificial. Plantea cómo llevarlo a la práctica con Python, un entorno de Anaconda/Conda, Jupyter Notebook para la experimentación y Keras/TensorFlow para entrenar el modelo en local. El resultado se guarda en formato .keras y se expone a través de una API construida con FastAPI, para que cualquier enfrentamiento entre dos selecciones se pueda consultar en segundos.

Con esta delimitación quedan planteados la situación problémica, la pregunta central y los problemas específicos. El apartado 1.2 traduce ese diagnóstico en objetivos verificables: el objetivo general enuncia el resultado que se persigue y los objetivos específicos responden, uno a uno, a los problemas específicos aquí identificados.

1.2. Objetivos

1.2.1. Objetivo general

El objetivo general es lo que se quiere llegar a tener al final del proyecto: a partir de las métricas reales de las selecciones, obtener un valor de probabilidad por cada enfrentamiento.

- **Objetivo general.** Desarrollar un simulador de enfrentamiento directo que, usando las estadísticas del dataset de la Copa Mundial de la FIFA 2026, estime la probabilidad de victoria entre dos selecciones mediante una red neuronal de una sola neurona con activación sigmoide, entrenada y ejecutada en local con Python.

1.2.2. Objetivos específicos

Cada objetivo es una fila con un verbo de acción concreto. Cuatro cubren el ciclo completo del trabajo: analizar, diseñar, implementar y desarrollar.

- **Analizar.** el dataset "FIFA World Cup 2026 Team Data" de Kaggle: descomprimir el archivo ZIP, revisar cada CSV encontrado e identificar las métricas útiles para el predictor.

- **Diseñar.** el vector de diferencias entre dos equipos y definir la arquitectura matemática de la neurona única (entrada, pesos, sesgo y función sigmoide).
- **Implementar.** el pipeline de entrenamiento en Python con Jupyter Notebook dentro de un entorno Conda, usando Keras/TensorFlow y scikit-learn para la estandarización de las características.
- **Desarrollar.** una API con FastAPI que cargue el modelo entrenado en formato .keras y devuelva la probabilidad de victoria para cualquier par de selecciones.

1.3. Alcances

El alcance marca hasta dónde llega el trabajo. Lo que el proyecto sí cubre es lo siguiente.

- Cubre las 48 selecciones clasificadas al Mundial 2026 que aparecen en el dataset de Kaggle.
- Estima la probabilidad de victoria del equipo A frente al equipo B a partir de diferencias de goles, posesión, disparos a puerta y edad promedio.
- Entrena y evalúa el modelo 100% en local, con entorno de Anaconda/Conda y Jupyter Notebook.
- Expone el modelo entrenado (formato .keras) mediante una API local de FastAPI.
- Documenta el análisis descriptivo del archivo teams.csv e incluye capturas de las primeras 10 filas en el Capítulo 2.

1.4. Límites

Los límites dicen lo que no se va a hacer. Son tan importantes como el alcance, porque evitan que el proyecto se desborde. Se presentan en filas con viñetas.

- no se desarrollará un sistema de apuestas ni se calcularán cuotas para casas de apuestas.
- no se entrenarán redes profundas ni modelos de más de una neurona; la solución se limita a la regresión logística.

•no se usará un historial de enfrentamientos reales previos entre selecciones; el modelo solo trabaja con las estadísticas agregadas del dataset.

•no se realizará despliegue en la nube ni servidores públicos; el entrenamiento y la inferencia se quedan en local.

•no se hará actualización automática de datos durante el Mundial ni predicción en tiempo real de partidos en vivo.

•no se predice el marcador exacto, los goleadores ni el campeón general del torneo; solo la probabilidad de victoria por partido.

1.5. Enfoque tecnológico

La idea es hacerse con herramientas que ya están instaladas y funcionan fuera de línea. Cada una cumple un papel distinto dentro del mismo flujo.

- **Jupyter Notebook.** entorno de experimentación, análisis exploratorio y visualización del dataset.
- **Anaconda / Conda.** creación y gestión del entorno virtual con sus dependencias (Python, pip y paquetes).
- **Python.** lenguaje principal para el pipeline de datos, el entrenamiento y la API.
- **Keras / TensorFlow.** implementación de la neurona única y guardado del modelo en formato .keras.
- **scikit-learn.** estandarización de características (StandardScaler) y métricas de evaluación.
- **pandas / numpy.** carga del CSV, generación de los pares de enfrentamiento y cálculo de las diferencias vectoriales.
- **FastAPI.** API local que carga el modelo final y responde la probabilidad de cada enfrentamiento.
- **Kaggle API.** descarga del dataset desde Kaggle para reproducir el entorno (opcional).

CAPÍTULO 2. MARCO PRÁCTICO

2.1. Análisis del dataset

El archivo descargado desde Kaggle llega comprimido en un ZIP llamado archive.zip. Al descomprimirlo se encuentra un solo archivo: teams.csv. Es un formato de texto plano que se abre sin problema desde pandas con una sola línea de código.

El CSV contiene 48 filas, una por cada selección clasificada, y 136 columnas. Su cabecera mezcla métricas en dos bloques: primero las estadísticas propias del equipo y luego las mismas métricas del rival (la versión "against"). De ahí salen columnas como goals, possession, shots_on_target, avg_age, cards_yellow y tarjetas rojas, además de decenas de indicadores de porteros y de juego defensivo.

De las 136 columnas no todas se necesitan para el simulador. Para el enfrentamiento directo bastan cuatro diferencias: goles, posesión, disparos a puerta y edad promedio. Lo demás sirve para el análisis descriptivo y para justificar por qué se eligieron esas cuatro.

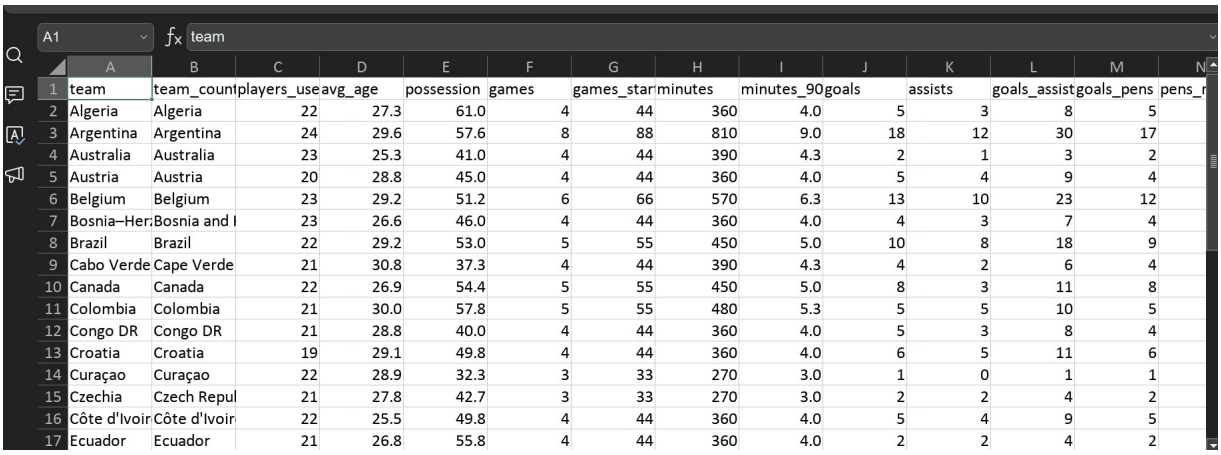
Tabla 1. Resumen descriptivo del dataset teams.csv.

URL DEL DATASEET https://www.kaggle.com/datasets/swaptr/fifa-wc-2026-teams

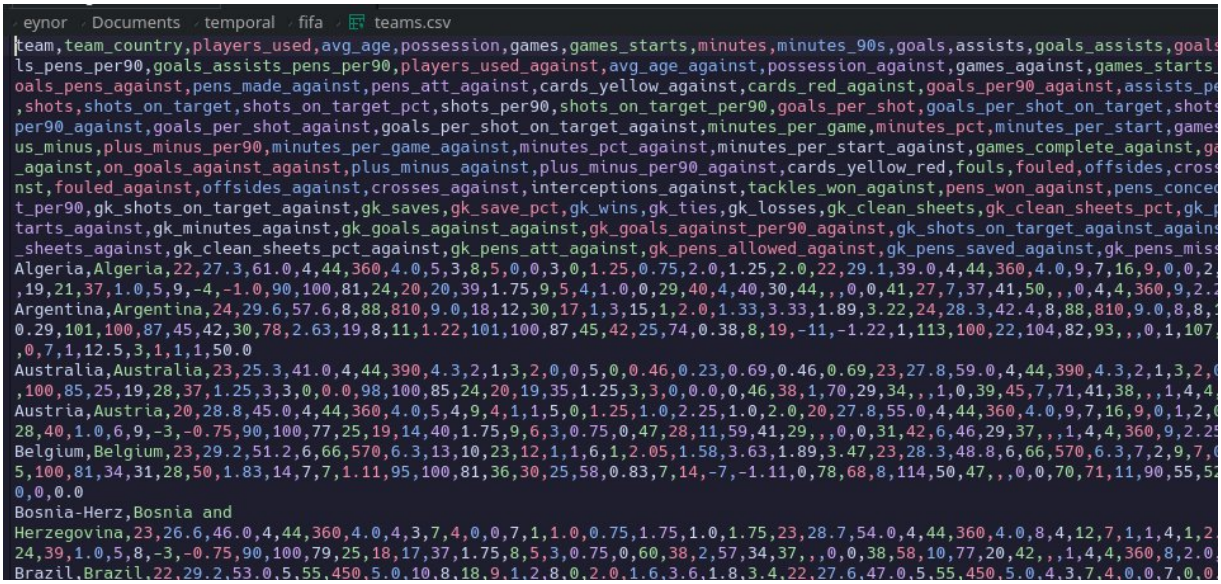
Métrica	Valor
Selecciones registradas	48
Goles promedio por selección	6.15
Posesión promedio	48.6%
Disparos a puerta promedio	18.46
Disparos totales promedio	53.7
Edad promedio del plantel	27.9 años
Tarjetas amarillas promedio	5.6
Tarjetas rojas promedio	0.31

Los promedios se calcularon sobre las 48 filas completas del archivo. Casi todas las selecciones jugaron el mismo número de partidos (entre 3 y 8), así que las diferencias entre equipos son comparables sin normalizar por cantidad de encuentros.

A continuación se muestra el espacio reservado para la captura de pantalla de las primeras 10 filas del CSV, tal como se solicita en la estructura del documento.



	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	team	team_count	players_use	avg_age	possession	games	games_star	minutes	minutes_90	goals	assists	goals_assist	goals_pens	pens_r
2	Algeria	Algeria	22	27.3	61.0	4	44	360	4.0	5	3	8	5	
3	Argentina	Argentina	24	29.6	57.6	8	88	810	9.0	18	12	30	17	
4	Australia	Australia	23	25.3	41.0	4	44	390	4.3	2	1	3	2	
5	Austria	Austria	20	28.8	45.0	4	44	360	4.0	5	4	9	4	
6	Belgium	Belgium	23	29.2	51.2	6	66	570	6.3	13	10	23	12	
7	Bosnia–Her	Bosnia and H	23	26.6	46.0	4	44	360	4.0	4	3	7	4	
8	Brazil	Brazil	22	29.2	53.0	5	55	450	5.0	10	8	18	9	
9	Cabo Verde	Cape Verde	21	30.8	37.3	4	44	390	4.3	4	2	6	4	
10	Canada	Canada	22	26.9	54.4	5	55	450	5.0	8	3	11	8	
11	Colombia	Colombia	21	30.0	57.8	5	55	480	5.3	5	5	10	5	
12	Congo DR	Congo DR	21	28.8	40.0	4	44	360	4.0	5	3	8	4	
13	Croatia	Croatia	19	29.1	49.8	4	44	360	4.0	6	5	11	6	
14	Curaçao	Curaçao	22	28.9	32.3	3	33	270	3.0	1	0	1	1	
15	Czechia	Czech Repul	21	27.8	42.7	3	33	270	3.0	2	2	4	2	
16	Côte d'Ivoir	Côte d'Ivoir	22	25.5	49.8	4	44	360	4.0	5	4	9	5	
17	Ecuador	Ecuador	21	26.8	55.8	4	44	360	4.0	2	2	4	2	



eynor / Documents / temporal / fifa / teams.csv

```
team,team_country,players_used,avg_age,possession,games,games_starts,minutes,minutes_90s,goals,assists,goals_assists,goals
ls_pens_per90,goals_assists_pens_per90,players_used_against,avg_age_against,possession_against,games_against,games_starts_
oals_pens_against,pens_made_against,pens_att_against,cards_yellow_against,cards_red_against,goals_per90_against,assists_pe
,shots,shots_on_target,shots_on_target_pct,shots_per90,shots_on_target_per90,goals_per_shot,goals_per_shot_on_target,shots
per90_against,goals_per_shot_against,goals_per_shot_on_target_against,minutes_per_game,minutes_pct,minutes_per_start,games
us_minus,plus_minus_per90,minutes_per_game_against,minutes_pct_against,minutes_per_start_against,games_complete_against,ga
_against,on_goals_against_against,plus_minus_against,plus_minus_per90_against,cards_yellow_red,fouls,fouled,offsides,cross
nst,fouled_against,offsides_against,crosses_against,interceptions_against,tackles_won_against,pens_won_against,pens_conced
t_per90,gk_shots_on_target_against,gk_saves,gk_save_pct,gk_wins,gk_ties,gk_losses,gk_clean_sheets,gk_clean_sheets_pct,gk_p
tarts_against,gk_minutes_against,gk_goals_against_against,gk_goals_against_per90_against,gk_shots_on_target_against_agains
_sheets_against,gk_clean_sheets_pct_against,gk_pens_att_against,gk_pens_allowed_against,gk_pens_saved_against,gk_pens_miss
Algeria,Algeria,22,27.3,61.0,4,44,360,4.0,5,3,8,5,0,0,3,0,1.25,0.75,2.0,1.25,2.0,22,29.1,39.0,4,44,360,4.0,9,7,16,9,0,0,2
,19,21,37,1.0,5,9,-4,-1.0,90,100,81,24,20,20,39,1.75,9,5,4,1.0,0,29,40,4,40,30,44,,,0,0,41,27,7,37,41,50,,,0,4,4,360,9,2.
Argentina,Argentina,24,29.6,57.6,8,88,810,9.0,18,12,30,17,1,3,15,1,2.0,1.33,3.33,1.89,3.22,24,28.3,42.4,8,88,810,9.0,8,8,
0.29,101,100,87,45,42,30,78,2.63,19,8,11,1.22,101,100,87,45,42,25,74,0.38,8,19,-11,-1.22,1,113,100,22,104,82,93,,,0,1,107
,0,7,1,12.5,3,1,1,1,50.0
Australia,Australia,23,25.3,41.0,4,44,390,4.3,2,1,3,2,0,0,5,0,0.46,0.23,0.69,0.46,0.69,23,27.8,59.0,4,44,390,4.3,2,1,3,2,
,100,85,25,19,28,37,1.25,3,3,0,0.0,98,100,85,24,20,19,35,1.25,3,3,0,0.0,0,46,38,1,70,29,34,,,1,0,39,45,7,71,41,38,,,1,4,4
Austria,Austria,20,28.8,45.0,4,44,360,4.0,5,4,9,4,1,1,5,0,1.25,1.0,2.25,1.0,2.0,20,27.8,55.0,4,44,360,4.0,9,7,16,9,0,1,2,
28,40,1.0,6,9,-3,-0.75,90,100,77,25,19,14,40,1.75,9,6,3,0.75,0,47,28,11,59,41,29,,,0,0,31,42,6,46,29,37,,,1,4,4,360,9,2.2
Belgium,Belgium,23,29.2,51.2,6,66,570,6.3,13,10,23,12,1,1,6,1,2.05,1.58,3.63,1.89,3.47,23,28.3,48.8,6,66,570,6.3,7,2,9,7,
5,100,81,34,31,28,50,1.83,14,7,7,1.11,95,100,81,36,30,25,58,0.83,7,14,-7,-1.11,0,78,68,8,114,50,47,,,0,0,70,71,11,90,55,5
0,0,0.0
Bosnia-Herz,Bosnia and
Herzegovina,23,26.6,46.0,4,44,360,4.0,4,3,7,4,0,0,7,1,1.0,0.75,1.75,1.0,1.75,23,28.7,54.0,4,44,360,4.0,8,4,12,7,1,1,4,1,2
24,39,1.0,5,8,-3,-0.75,90,100,79,25,18,17,37,1.75,8,5,3,0.75,0,60,38,2,57,34,37,,,0,0,38,58,10,77,20,42,,,1,4,4,360,8,2.0
Brazil,Brazil,22,29.2,53.0,5,55,450,5.0,10,8,18,9,1,2,8,0,2.0,1.6,3.6,1.8,3.4,22,27.6,47.0,5,55,450,5.0,4,3,7,4,0,0,7,0,0
```

2.1.1. Verificación de los archivos CSV del ZIP

Además del análisis descriptivo, se documenta la revisión completa del paquete de descarga. El archivo comprimido obtenido de Kaggle se descomprime en la carpeta de trabajo y su contenido queda registrado en la siguiente tabla, donde se indica, para cada archivo, qué se verificó y con qué evidencia queda respaldada esa revisión.

Tabla 2. Inventario de los archivos del ZIP y verificación realizada.

Archivo	Contenido verificado	Revisión realizada	Evidencia
archive.zip	**Paquete comprimido descargado de Kaggle con el dataset "FIFA World Cup 2026 Team Data".**	**Descomprimido en la carpeta de trabajo e inventario de su contenido.**	**Captura 1**
teams.csv	**48 filas (una por selección clasificada) y 136 columnas de métricas ofensivas, defensivas y de portería.**	**Recorrido íntegro del archivo: encabezado, primeras 10 filas, valores vacíos y distribución de partidos por equipo.**	**Captura 2**

Nota: Elaboración propia a partir de la revisión del archivo descargado de Kaggle (https://www.kaggle.com/datasets/swaptr/fifa-wc-2026-teams).

La verificación confirma que el dataset no incluye información partido a partido ni archivos adicionales con emparejamientos históricos, lo que confirma la necesidad de construir las etiquetas de forma sintética descrita en el apartado 1.1.3. A continuación se insertan las capturas de pantalla correspondientes al contenido del archivo comprimido y a las primeras filas de cada archivo CSV revisado, de modo que la verificación quede respaldada visualmente.

Figura 1. Contenido del archivo comprimido archive.zip.

Figura 2. Primeras filas del archivo teams.csv.

2.2. Pipeline de implementación propuesto

La lógica es corta y se puede seguir paso a paso en un notebook. Los pares de equipos se generan de forma sintética: se toma cada selección contra cada

selección distinta y se calcula el vector de diferencias. La etiqueta inicial es sencilla: gana el equipo que tenga más goles, a la espera de validarla con datos reales más adelante.

Fragmento 1. Pipeline de entrenamiento de la neurona única (Keras).

```
import pandas as pd
import numpy as np
import tensorflow as tf
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("teams.csv")

features = ['goals', 'possession', 'shots_on_target', 'avg_age']
X_diff, y_label = [], []

nations = df["team"].unique()
for i in range(len(nations)):
    for j in range(len(nations)):
        if i != j:
            team_a = df[df["team"] == nations[i]][features].values[0]
            team_b = df[df["team"] == nations[j]][features].values[0]

            diff = team_a - team_b
            X_diff.append(diff)
            label = 1 if (team_a[0] > team_b[0]) else 0
            y_label.append(label)

X_diff = np.array(X_diff)
y_label = np.array(y_label)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_diff)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=1,             input_shape=(len(features),),
activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_scaled, y_label, epochs=100, batch_size=8, verbose=1)

model.save("fifa_2026_h2h_neuron.keras")
print("Modelo guardado como fifa_2026_h2h_neuron.keras")
```

Terminado el entrenamiento, el archivo .keras queda guardado en local. Con FastAPI se define un solo endpoint: el cliente envía los nombres de dos selecciones y la API calcula la diferencia, la estandariza con el escalador guardado y devuelve la

probabilidad de victoria del primer equipo. Si el valor es igual o mayor a 0.5 gana el equipo A, si es menor gana el equipo B o empata.

2.3. Limpieza de datos

El archivo teams.csv no puede pasar directamente al entrenamiento del simulador: antes se requiere una etapa de limpieza que lo deje en condiciones de ser leído por la red neuronal. Esa etapa se implementa en el notebook de Google Colab Limpieza_FIFA2026_Colab.ipynb, que acompaña a este documento y ejecuta el mismo procedimiento sobre el archivo archive.zip descargado de Kaggle. El notebook carga el paquete comprimido, verifica su contenido, diagnostica los problemas de calidad del CSV y exporta el resultado depurado en los archivos teams_limpio.csv y variables_modelo.csv.

El diagnóstico del archivo teams.csv, con 48 selecciones y 136 variables, identificó cinco problemas que fue necesario corregir. Hay variables sin ningún dato registrado: las cuatro del desempate por penaltis (pens_won, pens_conceded, pens_won_against y pens_conceded_against) están vacías en las 48 filas. Además, dos variables de porcentaje de minutos son constantes, 18 grupos de variables contienen información idéntica (por ejemplo, games coincide con games_against y con gk_games), por lo que 24 variables son redundantes, y la variable team_country repite el nombre de la selección con otra grafía en cinco casos. Por último, dos variables de penaltis del portero tienen 33 de 48 valores nulos. A esto se suma que las selecciones disputaron entre 3 y 8 partidos, de modo que las métricas acumuladas no son comparables entre equipos: la correlación entre los goles y los partidos disputados es de 0,89. La Tabla 3 resume el diagnóstico y las decisiones tomadas en cada caso.

Tabla 3. Diagnóstico y decisiones de limpieza aplicadas al dataset teams.csv.

Hallazgo	Variables afectadas	Acción aplicada	Resultado
Variables sin datos	pens_won, pens_conceded, pens_won_against, pens_conceded_against (48 de 48 vacías)	Eliminación	136 a 132 variables
Variables constantes	minutes_pct, minutes_pct_against	Eliminación	132 a 130 variables
Variables duplicadas	18 grupos idénticos, 24 variables redundantes (games = games_against = gk_games)	Se conserva la primera de cada grupo	130 a 106 variables
Variable redundante	team_country, alias del nombre de la selección en 5 casos	Eliminación	106 a 105 variables
Valores nulos	gk_pens_save_pct y gk_pens_save_pct_against (33 de 48)	Imputación por la mediana	258 nulos a 0
Partidos no comparables	games entre 3 y 8 (correlación con goals de 0,89)	Variables por partido y por 90 minutos	Métricas comparables
Variables categóricas	team, los 48 nombres de las selecciones	Codificación por etiquetas y one-hot si aplica	0 variables de texto en los CSV

Nota. La limpieza se ejecuta en el notebook Limpieza_FIFA2026_Colab.ipynb.

Como resultado, el dataset depurado conserva las 48 selecciones con 105 variables útiles, más dos variables nuevas calculadas por partido (goles por partido y disparos a puerta por partido), y se guarda como teams_limpio.csv. Para el simulador de enfrentamiento directo se seleccionan las cuatro variables que forman el vector de diferencias del apartado 2.2, goles, posesión, disparos a puerta y edad promedio, junto con el número de partidos y los dos ratios por partido, lo que da un archivo de 48 filas y 8 variables que se exporta como variables_modelo.csv.

Como el modelo no admite valores de texto, la única variable categórica que permanece tras la limpieza, el nombre de la selección, se sustituye por un código entero mediante codificación por etiquetas: las 48 selecciones se numeran del 1 al 48 en orden alfabético y la equivalencia se registra en el archivo codigos_equipos.csv. Como no queda ninguna otra variable de baja cardinalidad, no se aplica codificación one-hot, que habría creado 48 columnas binarias adicionales sin aportar información. De esta forma, ni el dataset depurado ni el archivo del

modelo contienen letras: todas sus columnas son numéricas y pueden entrar directamente al escalador estándar y a la red neuronal del pipeline del apartado 2.2.

Fragmento 2. Limpieza del dataset teams.csv en el notebook de Google Colab.

```
# Diagnóstico: nulos, constantes y variables duplicadas
nulos = df.isna().sum()
constantes = [c for c in df.columns if df[c].nunique(dropna=True) <= 1]
numericas = df.select_dtypes(include=[np.number])
grupos = {}
for c in numericas.columns:
    clave = tuple(numericas[c].fillna(-999).round(6).tolist())
    grupos.setdefault(clave, []).append(c)
duplicadas = [g for g in grupos.values() if len(g) > 1]


# Limpieza: eliminar, imputar y normalizar
df_clean = df.drop(columns=constantes)                          # vacías y
constantes
a_eliminar = [c for g in duplicadas for c in g[1:]]           # se conserva
la primera
a_eliminar = [c for c in a_eliminar if c in df_clean.columns]
df_clean = df_clean.drop(columns=a_eliminar)                # redundantes
df_clean = df_clean.drop(columns=['team_country'])             # alias del
nombre
for c in df_clean.columns[df_clean.isna().any()]:
    df_clean[c] = df_clean[c].fillna(df_clean[c].median())  # imputación
df_clean['goals_per_game']          =          (df_clean['goals']          /
df_clean['games']).round(3)
df_clean['shots_on_target_per_game'] = (
    df_clean['shots_on_target'] / df_clean['games']).round(3)



# Codificación de variables categóricas: el nombre pasa a ser un código
entero
codigos = {nombre: i + 1 for i, nombre
           in enumerate(sorted(df_clean['team'].unique()))}
df_clean['team_codigo'] = df_clean['team'].map(codigos)
df_clean = df_clean.drop(columns=['team'])
diccionario    =    pd.DataFrame(sorted(codigos.items()),    columns=['team',
'team_codigo'])

# Selección de variables y exportación
features = ['goals', 'possession', 'shots_on_target', 'avg_age']
ratios = ['goals_per_game', 'shots_on_target_per_game']
variables = ['team_codigo', 'games'] + features + ratios
df_modelo = df_clean[variables].sort_values('team_codigo')
df_modelo = df_modelo.reset_index(drop=True)
df_clean.to_csv('teams_limpio.csv', index=False)
df_modelo.to_csv('variables_modelo.csv', index=False)
diccionario.to_csv('codigos_equipos.csv', index=False)
```

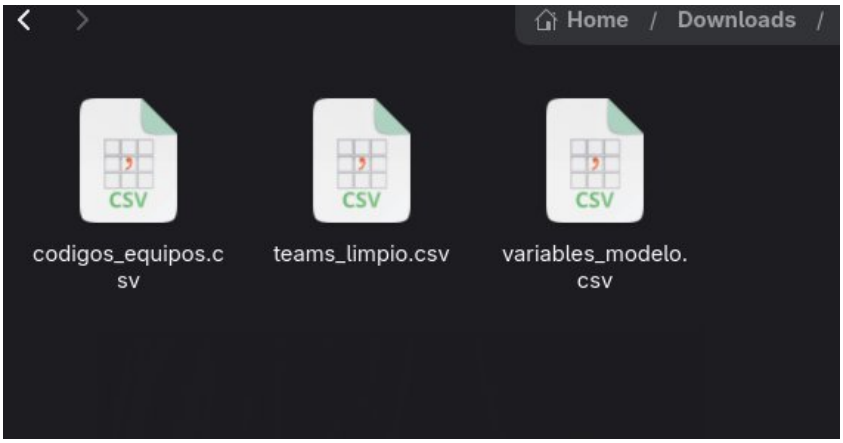
La verificación final del notebook confirma que el archivo resultante no contiene valores nulos ni variables de texto, mantiene las 48 selecciones sin duplicados y conserva la trazabilidad de cada decisión, ya que la celda de diagnóstico precede a la de limpieza y muestra las variables afectadas antes de transformarlas. El archivo variables_modelo.csv es el que se carga en el primer paso

del pipeline del apartado 2.2, de modo que la red neuronal recibe datos homogeneizados y comparables entre selecciones.

ANEXO

Archivo Con CSV GENRADOS

/Fifa.zip

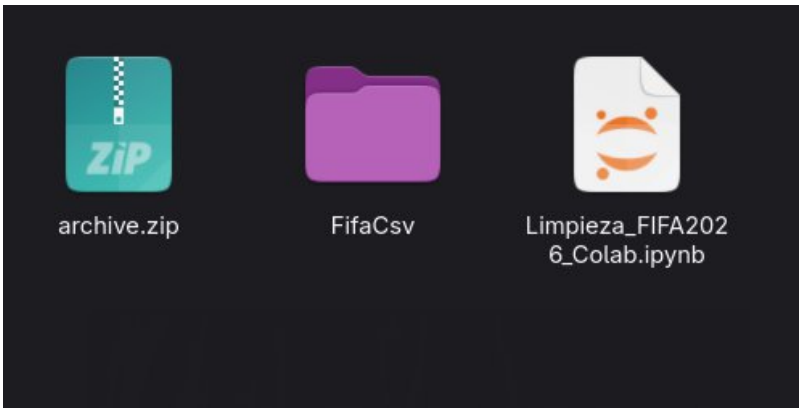


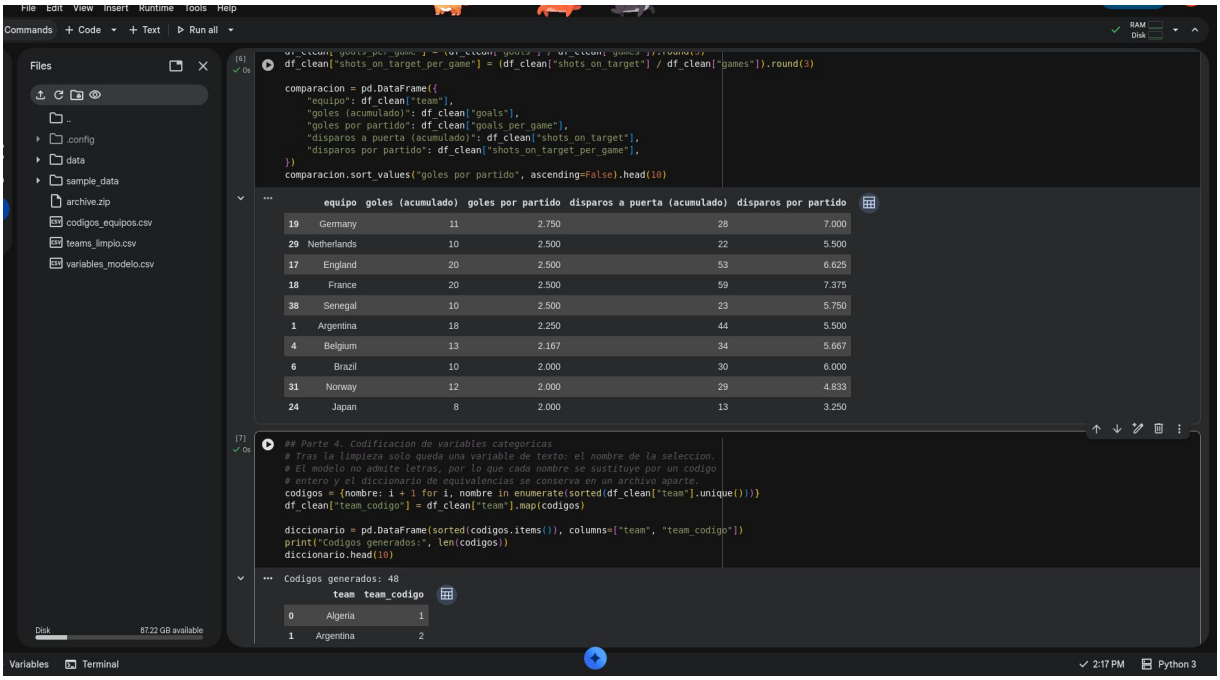
Codigo Limpieza

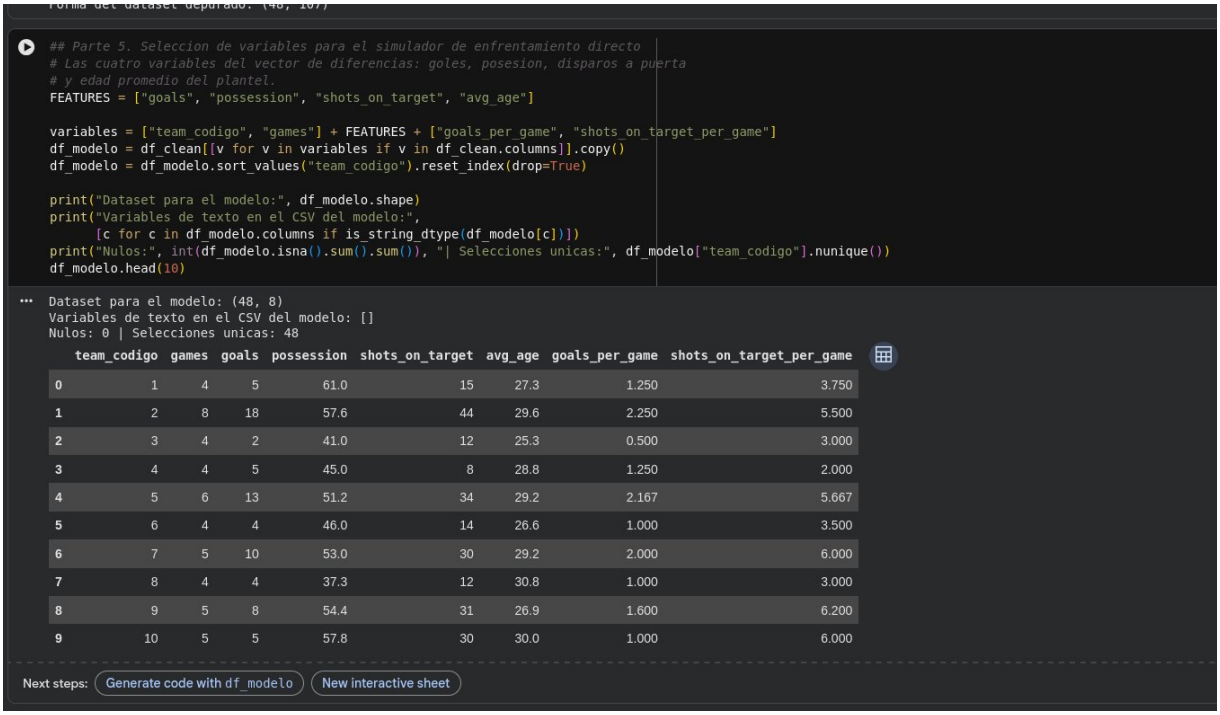
/DocumentColab

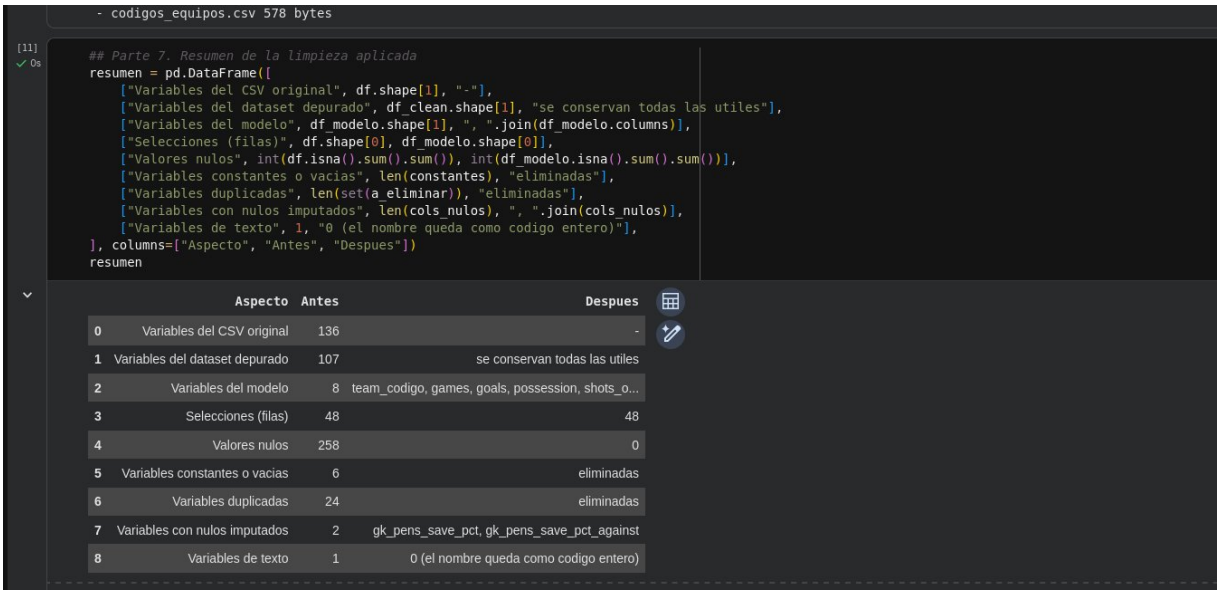
DATASET FIFA .zip

/archive.zip




File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Files
.config
data
sample_data
archive.zip
codigos_equipos.csv
teams_limpio.csv
variables_modelo.csv
Disk 87.22 GB available
df_clean["shots_on_target_per_game"] = (df_clean["shots_on_target"] / df_clean["games"]).round(3)
comparacion = pd.DataFrame({
"equipo": df_clean["team"],
"goles (acumulado)": df_clean["goals"],
"goles por partido": df_clean["goals_per_game"],
"disparos a puerta (acumulado)": df_clean["shots_on_target"],
"disparos por partido": df_clean["shots_on_target_per_game"],
})
comparacion.sort_values("goles por partido", ascending=False).head(10)
equipo goles (acumulado) goles por partido disparos a puerta (acumulado) disparos por partido
19 Germany 11 2.750 28 7.000
29 Netherlands 10 2.500 22 5.500
17 England 20 2.500 53 6.625
18 France 20 2.500 59 7.375
38 Senegal 10 2.500 23 5.750
1 Argentina 18 2.250 44 5.500
4 Belgium 13 2.167 34 5.667
6 Brazil 10 2.000 30 6.000
31 Norway 12 2.000 29 4.833
24 Japan 8 2.000 13 3.250
Parte 4. Codificacion de variables categoricas
Tras la limpieza solo queda una variable de texto: el nombre de la seleccion.
El modelo no admite letras, por lo que cada nombre se sustituye por un codigo
entero y el diccionario de equivalencias se conserva en un archivo aparte.
codigos = {nombre: i + 1 for i, nombre in enumerate(sorted(df_clean["team"].unique()))}
df_clean["team_codigo"] = df_clean["team"].map(codigos)
diccionario = pd.DataFrame(sorted(codigos.items()), columns=["team", "team_codigo"])
print("Codigos generados:", len(codigos))
diccionario.head(10)
Codigos generados: 48
team team_codigo
0 Algeria 1
1 Argentina 2
Variables Terminal
2:17 PM Python 3


Parte 5. Seleccion de variables para el simulador de enfrentamiento directo
Las cuatro variables del vector de diferencias: goles, posesion, disparos a puerta
y edad promedio del plantel.
FEATURES = ["goals", "possession", "shots_on_target", "avg_age"]
variables = ["team_codigo", "games"] + FEATURES + ["goals_per_game", "shots_on_target_per_game"]
df_modelo = df_clean[[v for v in variables if v in df_clean.columns]].copy()
df_modelo = df_modelo.sort_values("team_codigo").reset_index(drop=True)
print("Dataset para el modelo:", df_modelo.shape)
print("Variables de texto en el CSV del modelo:",
[c for c in df_modelo.columns if is_string_dtype(df_modelo[c])])
print("Nulos:", int(df_modelo.isna().sum().sum()), "| Selecciones unicas:", df_modelo["team_codigo"].nunique())
df_modelo.head(10)
Dataset para el modelo: (48, 8)
Variables de texto en el CSV del modelo: []
Nulos: 0 | Selecciones unicas: 48
team_codigo games goals possession shots_on_target avg_age goals_per_game shots_on_target_per_game
0 1 4 5 61.0 15 27.3 1.250 3.750
1 2 8 18 57.6 44 29.6 2.250 5.500
2 3 4 2 41.0 12 25.3 0.500 3.000
3 4 4 5 45.0 8 28.8 1.250 2.000
4 5 6 13 51.2 34 29.2 2.167 5.667
5 6 4 4 46.0 14 26.6 1.000 3.500
6 7 5 10 53.0 30 29.2 2.000 6.000
7 8 4 4 37.3 12 30.8 1.000 3.000
8 9 5 8 54.4 31 26.9 1.600 6.200
9 10 5 5 57.8 30 30.0 1.000 6.000
Next steps: Generate code with df_modelo New interactive sheet



```
- codigos_equipos.csv 578 bytes
```

```
[11]
✓ 0s
```

```
## Parte 7. Resumen de la limpieza aplicada
resumen = pd.DataFrame([
    ["Variables del CSV original", df.shape[1], "-"],
    ["Variables del dataset depurado", df_clean.shape[1], "se conservan todas las utiles"],
    ["Variables del modelo", df_modelo.shape[1], ", ".join(df_modelo.columns)],
    ["Selecciones (filas)", df.shape[0], df_modelo.shape[0]],
    ["Valores nulos", int(df.isna().sum().sum()), int(df_modelo.isna().sum().sum())],
    ["Variables constantes o vacias", len(constantes), "eliminadas"],
    ["Variables duplicadas", len(set(a_eliminar)), "eliminadas"],
    ["Variables con nulos imputados", len(cols_nulos), ", ".join(cols_nulos)],
    ["Variables de texto", 1, "0 (el nombre queda como codigo entero)"],
], columns=["Aspecto", "Antes", "Despues"])
resumen
```

	Aspecto	Antes	Despues
0	Variables del CSV original	136	-
1	Variables del dataset depurado	107	se conservan todas las utiles
2	Variables del modelo	8	team_codigo, games, goals, possession, shots_o...
3	Selecciones (filas)	48	48
4	Valores nulos	258	0
5	Variables constantes o vacias	6	eliminadas
6	Variables duplicadas	24	eliminadas
7	Variables con nulos imputados	2	gk_pens_save_pct, gk_pens_save_pct_against
8	Variables de texto	1	0 (el nombre queda como codigo entero)

BIBLIOGRAFÍA

Kaggle. (2026). FIFA World Cup 2026 Team Data. Recuperado de https://www.kaggle.com/datasets/swaptr/fifa-wc-2026-teams

Chollet, F. (2021). Deep Learning with Python (2.ª ed.). Manning Publications.

Documentación oficial de Keras. Recuperado de https://keras.io

Documentación oficial de FastAPI. Recuperado de https://fastapi.tiangolo.com

Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (3.ª ed.). O'Reilly Media.